

ACDAN: Adaptive Candidate and Verifier Allocation for Cost-Calibrated Test-Time Inference

Anonymous submission

Abstract

Test-time scaling improves language-model reasoning by sampling, searching, or verifying multiple candidate outputs, but its cost grows quickly and its confidence estimates are often unreliable. We study the problem of *adaptive candidate and verifier allocation*: given a task and a pool of candidate answers or tool actions, decide which candidates deserve additional compute, which evidence to trust, and when to stop, optimizing the accuracy–cost–latency–calibration frontier. We present ACDAN, an ablation-ready inference architecture that refines a soft candidate-action distribution with Differentiable Textual Optimization (DTO), optionally uses self-consistency evidence as an explicit controlled prior, prunes redundant multi-step tool plans through an agentic dependency graph, reuses high-inertia tool transitions from a separate fit split, and produces calibrated accept/abstain decisions through self-verification. The implementation includes neutral-order math candidate generation, an early-stopping adaptive self-consistency baseline, real-data loaders for GSM8K, MATH-500, AIME 2025, Omni-MATH, and BFCL, per-task artifacts, and scripts for Pareto analysis. The paper reports the exact protocol and table templates required for the final AAI-27 experiments; numeric benchmark cells are intentionally left to be filled only after the full real-backend matrix is run.

1 Introduction

Large language models increasingly spend extra inference compute by sampling multiple reasoning traces, reranking candidates with verifiers, or routing tasks to stronger models. These strategies are useful but blunt: self-consistency asks “should we sample more?”, verifier reranking asks “which candidate is best after scoring many candidates?”, and LLM routers ask “which model should answer?” ACDAN targets the candidate-level middle ground: *which candidates deserve compute or verification, and how much?*

We position ACDAN as adaptive test-time compute allocation, not as a claim of new intrinsic reasoning ability. The method operates over a finite action or candidate vocabulary. In math, this is answer selection over pre-generated candidate solutions. In tool use, this is tool/action selection over one or more planning steps. This framing enables direct

comparison with Chain-of-Thought greedy decoding, self-consistency, early-stopping self-consistency, Best-of- N with a PRM/verifier, reward-free selectors, and model-routing cascades.

The contribution is threefold:

- We formulate adaptive candidate/verifier allocation as a constrained test-time objective over expected accuracy, token/latency cost, verifier calls, and calibration.
- We implement ACDAN as a reproducible inference system with DTO, explicit self-consistency evidence controls, graph/inertia efficiency modules for multi-step tools, and calibrated verification.
- We provide the approval protocol for AAI-27: neutral-order candidate files, adaptive baselines, full math and BFCL matrices, per-task logs, and Pareto analysis scripts. We do not fill benchmark cells until real runs complete.

2 Related Work and Taxonomy

Table 1 summarizes the closest method families. ACDAN is most closely related to adaptive self-consistency and verifier-based reranking, but it differs by making candidate-level evidence, verifier budget, and calibration jointly visible in the same experiment protocol.

3 Problem Formulation

For task x , let $\mathcal{C}(x) = \{c_1, \dots, c_K\}$ be candidate answers, tool calls, or action templates. A test-time policy chooses a possibly adaptive sequence of operations z : sample, score with a policy prior, query a verifier or PRM, accept, abstain, or stop. Let $a(z)$ be the selected final candidate and y the reference answer or tool trace. We optimize the frontier

$$\max_{\pi} \mathbb{E}[\text{Acc}(a(z), y)] - \lambda_t \mathbb{E}[\text{Tokens}(z)] - \lambda_\ell \mathbb{E}[\text{Latency}(z)] - \lambda_v \mathbb{E}[\text{Verifiers}(z)] \quad (1)$$

subject to a coverage constraint $\mathbb{P}[\pi \text{ answers}] \geq \rho$. This paper evaluates methods by the empirical Pareto frontier rather than a single scalarization.

Candidate diagnostics. For math candidate files, every run reports oracle@K, always-first accuracy, always-last accuracy, and highest-count accuracy. These controls detect candidate-order leakage and self-consistency plurality bias.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Table 1: Taxonomy of methods near ACDAN. The table defines the baselines needed for the final AAI-27 experiment matrix.

Family	Representative methods	Main decision
Self-consistency	SC-CoT (Wang et al. 2022)	vote over samples
Adaptive SC	ESC/DSC/RASC/ReASC (Li et al. 2024; Wang et al. 2024; Wan et al. 2025; Kim et al. 2026)	when to stop sampling
Verifier / PRM	PRM800K, Math-Shepherd, GenRM (Lightman et al. 2023; Wang et al. 2023; Zhang et al. 2024)	score/rerank candidates
Reward-free selection	Self-Certainty (Kang et al. 2025)	select without scores
Routing / cascade	FrugalGPT, RouteLLM, ZOOTOE (Chen, Zaharia, and Zou 2023; Ong et al. 2024; Lu et al. 2024)	choose model/runner
ACDAN	this work	allocate candidates

Algorithm 1 ACDAN Inference

- 1: Encode task x and candidate/action set $\mathcal{C}(x)$; compute latent quality $q(u)$
- 2: Produce prior logits $L_0 = f_\theta(x, \mathcal{C})$
- 3: Refine logits by DTO with PRM target sharpness conditioned on $q(u)$
- 4: Decode draft actions $d_h = \arg \max_v \text{softmax}(L_T/\tau)_{hv}$
- 5: Overlay high-confidence inertia actions fit from train/dev traces only
- 6: Score executed actions, compute Net Information Gain, build dependency graph
- 7: Prune dead steps for cost accounting; never prune final step
- 8: Verify executed plan, calibrate confidence, then accept or abstain
- 9: **return** selected candidate/tool trace, confidence, cost metrics

ACDAN objective. The core model emits candidate logits $L_0 \in \mathbb{R}^{H \times V}$ for horizon H and vocabulary size V . DTO optimizes logits L with $P = \text{softmax}(L/\tau)$:

$$J(L) = -w_\ell \frac{1}{H} \sum_{h,v} P_{hv} \log p_{0,hv} - \alpha R_{\text{prm}}(P) + \beta R_{\text{rep}}(P) - \gamma H_{\text{dep}}(P) - \delta_{\text{sc}} S_{\text{sc}}(P), \quad (2)$$

where S_{sc} is an optional standardized log-count prior. In the main math setting $\delta_{\text{sc}} = 0$ and candidate counts are hidden from prompts and PRM scoring. Count evidence is only enabled in a named ablation (none, prompt, prm, dto, all).

4 Method

Latent-quality-conditioned PRM. ACDAN includes a lightweight latent reasoning block that maps the encoded task to a state u and scalar quality $q(u) \in [0, 1]$. The implementation uses this quality as a gate on verifier sharpness rather than as hidden-state adaptation of the underlying LLM. Given raw PRM rewards r_{hv} , DTO receives the target

$$t_{hv} = \text{softmax}_v(s(u)r_{hv}), \quad s(u) = s_0 \max\{0.1, 1 + \eta_q(2q(u) - 1)\}, \quad (3)$$

so higher-quality latent states produce a more decisive PRM gradient and lower-quality states produce a flatter, more conservative target. The same latent summary also feeds the calibration probe, but the method does not update the base

Algorithm 2 Adaptive Self-Consistency Baseline

- 1: Initialize vote counts over candidate answers
- 2: **for** $i = 1$ to N_{max} **do**
- 3: Add the next sample answer (or replay grouped candidate counts)
- 4: $q \leftarrow$ current plurality count divided by samples used
- 5: **if** $i \geq N_{\text{min}}$ and $q \geq \theta_{\text{asc}}$ **then**
- 6: stop early
- 7: **end if**
- 8: **end for**
- 9: **return** plurality answer, samples used

LLM’s transformer weights, activations, or KV cache at inference time.

Graph and inertia modules are evaluated on multi-step tools. They should not be claimed to improve GSM8K/MATH accuracy because those are one-step answer-selection tasks. BFCL correctness in this implementation is tool-name sequence correctness; argument-level claims require official AST/executable evaluation.

5 Implementation and Reproducibility

The repository implements:

- neutral-order candidate generation (`-order sample`) and legacy candidate reordering;
- ACDAN, CoT, SC, adaptive SC, and BoN+PRM baselines in one runner;
- per-task artifact saving through `-save-per-task`;
- data setup for GSM8K (Cobbe et al. 2021), MATH-500/MATH (Hendrycks et al. 2021), AIME 2025, Omni-MATH (Omni-MATH Authors 2024), and BFCL (Gorilla Team 2024);
- Pareto CSV/SVG collection through `scripts/collect_pareto.py`.

The data setup script reads HF_TOKEN from `.env` through `load_env()`, but the core package remains offline-friendly. All reported final runs must store the exact result JSON, per-task rows, model IDs, candidate file hashes, seeds, and command lines.

6 Experimental Protocol

The final submission should use the matrix in Table 2. All cells marked “TBD” must be filled from real result JSON files, not edited manually from ad hoc logs.

Table 2: Required final experiment matrix. $N \in \{1, 2, 4, 8, 16\}$ for SC, ASC, and BoN; CoT is $N = 1$. Report mean $\pm$ std over seeds 0, 1, 2.

Suite	Datasets	Methods	Required diagnostics	Status
Math sanity	GSM8K	CoT, SC, ASC, BoN, ACDAN	oracle@K, first, highest-count	TBD
Math hard	MATH-500, AIME 2025	CoT, SC, ASC, BoN, ACDAN	parser/evidence ablation	TBD
Math broad	Omni-MATH subset/full	CoT, SC, ASC, BoN, ACDAN	accuracy-cost Pareto	TBD
Tools	BFCL	CoT, SC, ASC, BoN, ACDAN + ablations	graph/inertia cost, per-task logs	TBD
Tool correctness	BFCL official	final predictions	AST/executable eval	future/TBD
Agentic	tau2/GAIA	executor-based methods	final answer success	future

Placeholder for
results/approval/pareto.svg. The final
figure plots accuracy vs mean token surrogate or real
prompt tokens for CoT, SC, ASC, BoN, and ACDAN
across N .

Figure 1: Accuracy–cost Pareto curve template. ACDAN is approval-ready only if it improves the frontier against SC, ASC, and BoN at matched cost or matched accuracy on at least one primary dataset.

The main paper should include two final tables and one figure:

1. Table 3: accuracy/cost/calibration at $N = 8$.
2. Table 4: ACDAN ablations and evidence attribution.
3. Figure 1: accuracy vs cost Pareto curve over N .

7 Approval Criteria

We will approve the method for the final AAI-27 submission only if all of the following are satisfied:

- math candidates are generated or rewritten with neutral order;
- ASC is included as an adaptive-compute baseline;
- main math and BFCL matrices run with per-task artifacts;
- the Pareto plot shows ACDAN on or above the frontier versus SC, ASC, and BoN for at least one primary dataset;
- evidence ablations show whether gains come from prompt counts, PRM count bonus, DTO count prior, or none;
- BFCL argument-level claims are made only with official AST/executable evaluation.

8 Limitations

Current limitations are explicit. First, math candidate selection is bounded by oracle@K; if no candidate is correct, no selector can recover. Second, ACDAN’s current PRM/LLM-as-verifier path may still score many candidates; verifier-call reductions must be demonstrated empirically rather than assumed. Third, graph and inertia are efficiency modules for multi-step tool traces and are not expected to improve one-step math accuracy. Fourth, BFCL conversion preserves gold argument metadata, but the default runner judges only tool-name sequences; official AST/executable evaluation is

required before claiming full function-calling correctness. Fifth, latent reasoning gates PRM sharpness through a scalar quality score; it does not perform hidden-state test-time training of the underlying LLM. Sixth, tau2/GAIA raw snapshots are prepared, but no final-answer executor is integrated yet. Seventh, several fast-moving related-work citations and dataset versions must be re-verified before camera-ready submission.

9 Conclusion

ACDAN reframes the project as adaptive candidate and verifier allocation for cost-calibrated test-time inference. The repository now contains the key submission gates: neutral candidate order controls, an adaptive self-consistency baseline, full-matrix scripts with per-task artifacts, Pareto analysis, and manuscript tables that require real experimental evidence before claims are filled. The remaining scientific question is empirical: whether ACDAN improves the accuracy–cost–calibration frontier against SC, ASC, and BoN on the prepared math and BFCL benchmarks.

Ethical Statement

ACDAN targets more efficient and better-calibrated test-time inference. Better calibration and abstention can reduce confident tool-use errors, but neither is a correctness guarantee. Candidate/verifier allocation could also make automated agents cheaper to deploy at scale, so high-stakes use requires conservative thresholds, domain-specific verification, and human oversight.

Table 3: Main real-backend results template. Fill only from results/approval/*.json.

Dataset	Method	Acc $\uparrow$	ECE $\downarrow$	Tok. $\downarrow$	Lat. $\downarrow$	Samples $\downarrow$	Verify $\downarrow$	Oracle@K	HighCnt
GSM8K	CoT	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GSM8K	SC@8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GSM8K	ASC@8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GSM8K	BoN@8	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GSM8K	ACDAN	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
MATH-500	ACDAN	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
AIME 2025	ACDAN	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
BFCL	ACDAN	TBD	TBD	TBD	TBD	TBD	TBD	NA	NA

Table 4: Ablation and evidence attribution template. Graph/inertia claims should be interpreted on BFCL/tool tasks, not one-step math.

Dataset	Variant	Acc $\uparrow$	ECE $\downarrow$	Tok. $\downarrow$	Verify $\downarrow$	Dead $\uparrow$	Inertia $\uparrow$
BFCL	ACDAN full	TBD	TBD	TBD	TBD	TBD	TBD
BFCL	no DTO	TBD	TBD	TBD	TBD	TBD	TBD
BFCL	no graph	TBD	TBD	TBD	TBD	TBD	TBD
BFCL	no inertia	TBD	TBD	TBD	TBD	TBD	TBD
BFCL	no verification	TBD	TBD	TBD	TBD	TBD	TBD
GSM8K	evidence none	TBD	TBD	TBD	TBD	NA	NA
GSM8K	evidence prompt	TBD	TBD	TBD	TBD	NA	NA
GSM8K	evidence prm	TBD	TBD	TBD	TBD	NA	NA
GSM8K	evidence dto	TBD	TBD	TBD	TBD	NA	NA
GSM8K	evidence all	TBD	TBD	TBD	TBD	NA	NA

References

- Chen, L.; Zaharia, M.; and Zou, J. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.
- Gorilla Team. 2024. Berkeley Function Calling Leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>. Accessed 2026-06-28; verify citation metadata before submission.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874.
- Kang, M.; et al. 2025. Scalable Best-of-N Selection for Large Language Models via Self-Certainty. Verify full author list before submission, arXiv:2502.18581.
- Kim, J.; Yang, N.; Min, K.; and Jung, K. 2026. Reliability-Aware Adaptive Self-Consistency for Efficient Sampling in LLM Reasoning. arXiv:2601.02970.
- Li, Y.; et al. 2024. Escape Sky-high Cost: Early-stopping Self-Consistency for Multi-step Reasoning. Verify full author list before submission, arXiv:2401.10480.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s Verify Step by Step. arXiv:2305.20050.
- Lu, K.; Yuan, H.; Lin, R.; Lin, J.; Yuan, Z.; Zhou, C.; and Zhou, J. 2024. Routing to the Expert: Efficient Reward-guided Ensemble of Large Language Models. In *Proceedings of NAACL*.
- Omni-MATH Authors. 2024. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models. Verify full author list before submission, arXiv:2410.07985.
- Ong, I.; Almahairi, A.; Wu, V.; Chiang, W.-L.; Wu, T.; Gonzalez, J. E.; Kadous, M. W.; and Stoica, I. 2024. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665.
- Wan, Z.; et al. 2025. Leveraging Reasoning Paths for Efficient LLM Sampling. In *Proceedings of NAACL*. Reasoning-Aware Self-Consistency; verify full author list before submission.
- Wang, P.; et al. 2023. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. Verify full author list before submission, arXiv:2312.08935.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.
- Wang, X.; et al. 2024. Make Every Penny Count: Difficulty-Adaptive Self-Consistency for Cost-Efficient Reasoning. Verify full author list before submission, arXiv:2408.13457.
- Zhang, L.; et al. 2024. Generative Verifiers: Reward Model-

ing as Next-Token Prediction. Verify full author list before submission, arXiv:2408.15240.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace **ONLY** the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this `.tex` file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) **yes**
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) **yes**
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) **yes**

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) **yes**

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) **partial**
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) **partial**
- 2.4. Proofs of all novel claims are included (yes/partial/no) **NA**
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) **partial**
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) **partial**
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) **partial**
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) **yes**

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) **yes**

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) **yes**
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) **NA**
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) **NA**
- 3.5. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are accompanied by appropriate citations (yes/no/NA) **partial**
- 3.6. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are publicly available (yes/partial/no/NA) **yes**
- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) **NA**

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) **yes**

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting

the final parameter setting (yes/partial/no/NA) [partial](#)

- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [yes](#)
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [yes](#)
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [yes](#)
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) [partial](#)
- 4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [yes](#)
- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [partial](#)
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [yes](#)
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [yes](#)
- 4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) [yes](#)
- 4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) [no](#)
- 4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [partial](#)